Pathfinder Team

Pilone & Milea Book Assignment

Problem 1:

The two major concerns of any software project are time and quality. Time refers to delivering the project within the scheduled timeline, while quality focuses on ensuring that the software meets user needs and is reliable. I believe quality is more important because software that doesn't function as intended or has major flaws can damage a company's reputation and result in high costs for bug fixes. Complete functionality fits with both concerns, as incomplete functionality can impact quality, while attempting to implement all desired features may cause delays, affecting time.

Problem 2:

In Agile, the four main phases in every iteration are planning, design, development, and testing. I don't believe any of these can be done only at the start and skipped in later iterations, as Agile relies on continuous feedback and adaptation. Repeating these phases ensures the software evolves to meet user requirements and helps uncover and resolve issues iteratively. Skipping them in later iterations would likely reduce flexibility and lead to missed opportunities for improvement.

Problem 3:

The main phases in the Waterfall method are requirements gathering, design, implementation, testing, and maintenance. Waterfall differs from Agile in that each phase is

completed before moving to the next, without returning to previous stages. Additional phases in Waterfall include detailed documentation and maintenance planning, which are less emphasized in Agile. These phases can be necessary for highly regulated environments where extensive documentation is required, such as in government or medical software. For instance, using Agile in a government project might still require detailed upfront documentation like in Waterfall.

Problem 4:

- What is a “user story”?

A user story is a simple description of a feature or functionality written from the user's perspective.

- What is “blueskying”?

“Blueskying” is the process of brainstorming creative or ambitious ideas without worrying about limitations.

- What are four things that user stories SHOULD do?

User stories should capture user needs, be concise, focus on value, and be testable.

- What are three things that user stories SHOULD NOT do?

User stories should not be too detailed, specify technical solutions, or be too broad.

Problem 5:

- All assumptions are bad, and no assumption is a “good” assumption.

I disagree because certain assumptions are unavoidable and can help move the project forward. However, assumptions must be clearly communicated and validated as soon as possible to prevent issues later on.

- A “big” user story estimate is a “bad” user story estimate.

I agree, as large estimates often indicate that the user story is too complex or not well understood. Breaking it down into smaller, more manageable pieces ensures better understanding and progress tracking.

Problem 6:

You can dress me up as a use case for a formal occasion: **User story**

The more of me there are, the clearer things become: **User Story**

I help you capture EVERYTHING: **Blueskying**

I help you get more from the customer: **Role playing**

In court, i'd be admissible as first hand evidence: **Observation**

Some people say I'm arrogant, but really i'm just about confidence: **Estimate**

Everyone's involved when it comes to me: **Blueskying**

Problem 7:

A better than best-case estimate refers to estimating a time or cost even more optimistically than the best-case scenario, which assumes everything will go perfectly. This kind of estimate is overly optimistic and often leads to underestimating the time needed.

Problem 8:

The best time to inform your customer that you cannot meet the delivery schedule is as soon as you realize there's a delay. This allows time to discuss alternatives or adjustments. While

difficult, addressing delays early demonstrates responsibility and transparency, creating the best environment for problem solving the delays.

Problem 9:

In general I think branching in software configuration is good practice however if done incorrectly it can really handicap yourself. It's useful when developing features independently, allowing team members to work in parallel without disrupting the main codebase. However, it can definitely be problematic if branches diverge too far from the main code, leading to complex merges. A scenario where branching is good is when different teams work on separate features simultaneously as this allows both teams to develop the features unhindered by any errors, issues, or differing code that the other team may bring up. However, it's important to regularly merge changes back into the main branch to avoid conflicts.

Problem 10:

I've used Gradle in development. Its strengths include dependency management and automation of tasks like testing and packaging. However, it can be slow with large projects and has a bit of a learning curve.